

TicketFlow — Documentación Técnica de Despliegue

Resolución de incidencias críticas, sincronización de entornos y puesta en producción

1. Resumen Ejecutivo del Estado del Sistema

Esta documentación detalla el proceso técnico de diagnóstico, resolución de errores de configuración y despliegue exitoso de la arquitectura completa del sistema **TicketFlow**. El entorno de producción final se encuentra completamente integrado y operativo en la nube, garantizando la persistencia y la seguridad en el intercambio de datos.

Estado Final Verificado: Circuito cerrado y operativo. El Front-end (Vercel) se comunica de forma exitosa con el Backend (Spring Boot en Render), el cual procesa la lógica de negocio y gestiona las solicitudes de manera segura sobre la base de datos relacional (PostgreSQL en Render).

2. Arquitectura y Enlaces de Entornos

El sistema se distribuye en una infraestructura desacoplada compuesta por los siguientes servicios activos:

Componente	Tecnología Principal	Plataforma de Hosting	Rama de Seguimiento (Git)
Front-End	JavaScript / React	Vercel	premaster (Producción Automatizada)
Back-End	Java / Spring Boot	Render	premaster
Base de Datos	PostgreSQL	Render (Cloud)	Persistente (Modo Update)

3. Bitácora de Errores Críticos Resolvidos

3.1. Corrección del Mecanismo de Inicialización de BD (Hibernate)

Problema: El backend se encontraba configurado en modo de inicialización destructivo (`create` o `create-drop`). Cada vez que el contenedor de Render se reiniciaba por inactividad o actualización de código, se destruían todas las tablas y registros preexistentes en PostgreSQL.

Solución: Se modificó el archivo de propiedades del entorno para asegurar que Hibernate únicamente valide y actualice la estructura existente sin alterar los registros vivos introducidos por los usuarios o scripts de migración.

```
# Ubicación: src/main/resources/application.properties
spring.jpa.hibernate.ddl-auto=update
```

3.2. Destrabe del Pipeline de Integración Continua (Render Deadlock)

Problema: El panel de Render experimentó un bloqueo en la cola de procesamiento interno (estado estático en *"Building"* seguido por un congelamiento en *"Canceling Deployment"*). Las peticiones manuales sucesivas encolaron procesos huérfanos.

Solución Técnica Aplicada:

- Inyección de una variable de entorno de control (REBOOT_FORCE=1) para obligar al backend de Render a invalidar la asignación previa del contenedor de Docker.
- Aplicación de un comando de detención de emergencia forzada mediante la acción avanzada **"Suspend Service"**. Una vez que el estado mutó exitosamente a *Suspended*, se procedió a la reactivación mediante **"Resume Service"**, forzando la descarga limpia de dependencias Maven mediante la instrucción *"Clear Cache and Deploy"*.

3.3. Corrección del Enrutamiento y "Branch Tracking" en Vercel

Problema: El Front-end en producción apuntaba por defecto al tracking de la rama *main*, la cual contenía una compilación desactualizada de las URLs de consumo de la API, provocando rechazos de autenticación, mientras que los despliegues de la rama activa se alojaban únicamente en URLs de *Preview*.

Solución: Se reconfiguró el flujo de Git en la consola administrativa de Vercel (*Settings -> Environments -> Production -> Edit Branch Tracking*), redirigiendo el mapeo de producción definitivo a la rama activa *premaster*. Con esta acción, el ciclo de vida de CI/CD quedó automatizado ante futuros pushes.

4. Diagnóstico de Pruebas de Endpoints (QA Report)

Durante el ciclo final de aseguramiento de calidad antes de la puesta en marcha, se ejecutaron pruebas cruzadas entre el cliente API de escritorio (Postman) y la interfaz de usuario en el navegador (Vercel DevTools):

4.1. Módulo de Autenticación y Registro de Usuarios

Se validó la consistencia del endpoint `POST /api/auth/register`. Inicialmente se detectó una incidencia aislada de error **500** provocado por desajustes en el payload (ausencia de asignación de rol por defecto al registrar desde el cliente web).

Resultado: Corrección verificada exitosamente. El flujo web impacta correctamente las credenciales en PostgreSQL y los usuarios pueden iniciar sesión (Login) obteniendo su correspondiente Token JWT (Bearer Token) en la URL oficial de Vercel.

4.2. Comportamiento de Módulos Protegidos (Eventos y Tickets)

Las pruebas de integración en Postman confirmaron el correcto funcionamiento del Backend ante las operaciones restringidas de creación de eventos y compra de tickets, siempre que se adjunte la cabecera correspondiente:

```
Authorization: Bearer <JWT_TOKEN>
```

Se observó una limitación en la interfaz de usuario web actual relacionada con el arrastre y persistencia del estado del token en el Local Storage entre transiciones de componentes de React. Esta brecha ha sido documentada como una tarea prioritaria de ajuste exclusivo en las capas del Front-end, garantizando que la API del Backend permanezca inalterable por su óptimo rendimiento actual.

5. Conclusión Técnica

La infraestructura actual cumple rigurosamente con los requisitos de la cátedra de Gestión de Tecnologías de la Información de la **Universidad Nacional de José C. Paz (UNPAZ)**, exponiendo una arquitectura moderna, segura, basada en tokens de autenticación y persistencia de datos en la nube lista para su evaluación formal.